STM32 timer and PWM

Timers

- A STM32L47xxx MCU SOC

  16x timers:

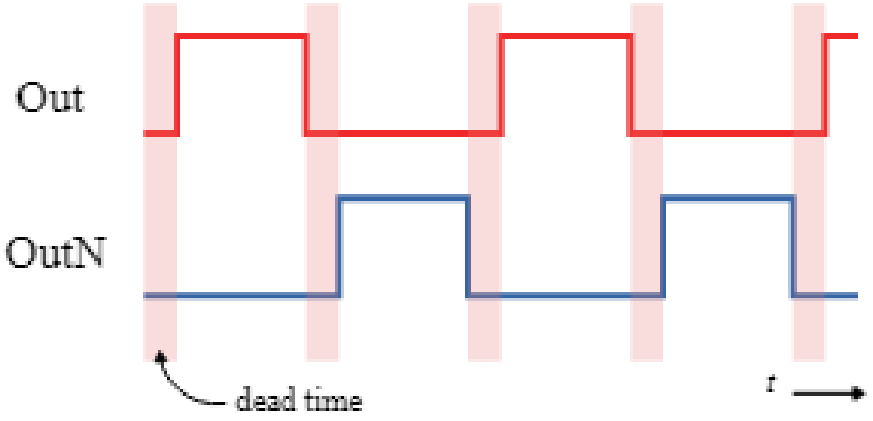
  - 2x 16-bit advanced motor-control,
  - 2x 32-bit and 5x 16-bit general purpose,
  - 2x 16-bit basic,
  - 2x low-power 16-bit timers (available in Stop mode),
  - 2x watchdogs,
  - SysTick timer

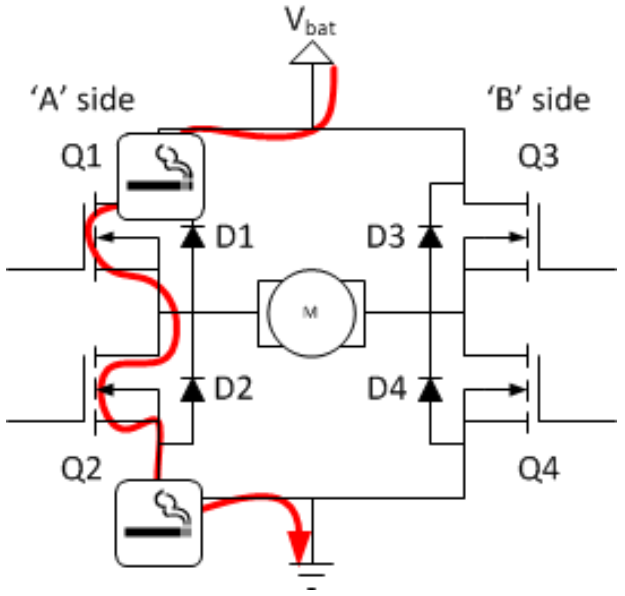
PWM waveforms are controlled using Timers

- The advanced-control timers (TIM1/TIM8) consist of a 16-bit auto-reload register (ARR) driven by a programmable prescaler (PSC).
- It may be used for a variety of purposes, including measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare, PWM, complementary PWM with dead-time insertion).
  - Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the Reset and Clock Control (RCC) clock controller.

What is “complementary PWM with dead-time insertion”?

- To ensure safe operation we put some duration between closing of one and opening other component. This time is usually called dead time in PWM context.
- Application: H-Bridge control





STM32 PWM Frequency

- In various applications, we'll be in need to generate a PWM signal with a specific frequency.
  - In servo motor control, LED drivers, motor drivers, and many more situations where we'll be in need to set your desired frequency for the output PWM signal.
  - The PWM period ($1/F_{PWM}$) is defined by the following parameters:
  - ARR value, the Prescaler(PSC) value, and the internal clock itself which drives the timer module CLK.

$$F_{PWM} = \frac{F_{CLK}}{(ARR + 1) \times (PSC + 1)}$$

STM PWM Duty Cycle

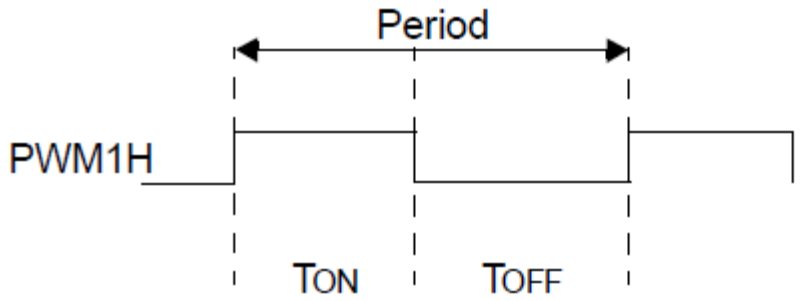
- In normal settings, assuming you're using the timer module in PWM mode and generating PWM signal in edge-aligned mode up-counting configuration.
  - The duty cycle percentage is controlled by changing the value of the capture control register (CCRx). And the duty cycle equals (CCRx/ARR) [%].

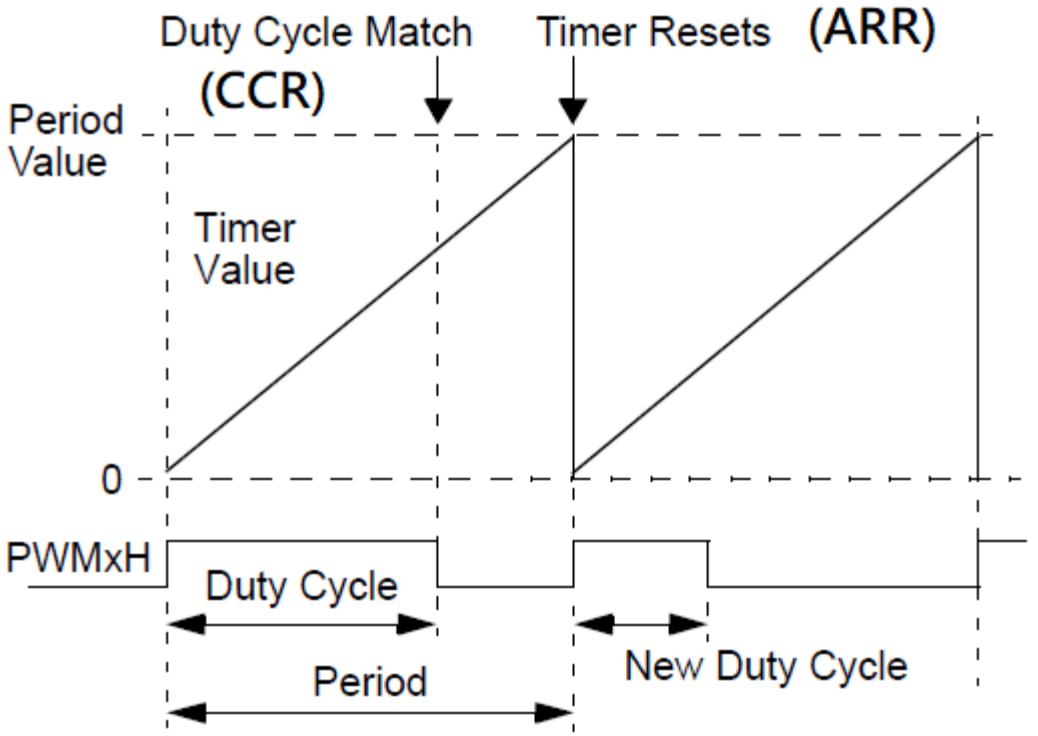
$$DutyCycle_{PWM}[\%] = \frac{CCRx}{ARRx}[\%]$$

Duty Cycle:

$$DutyCycle_{PWM}[\%] = \frac{CCRx}{ARRx}[\%]$$

- CCR: Capture Control Register
- ARR: Auto-Reload Register





STM32 PWM resolution

- One of the most important properties for a PWM signal is the **resolution**. It's the number of discrete duty cycle levels that we can set it to.
  - This number determines how many steps the duty cycle can take until it reaches the maximum value.
  - That is a 10 bit resolution means that you can have 1024 steps from zero to full power. Consider the example in which PWM is used to control the speed of a DC Motor. Using a PWM of 10 bit resolution we can have 1024 different speed for DC Motor.
- This is the STM32 PWM resolution formula that can be used to calculate the resolution of the PWM signal at a specific frequency or even the opposite.
  - If we're willing to get a 10-Bit resolution PWM signal, what should the frequency be in order to achieve this? And so on..

1. Resolution Calculation Equation

$$Resolution_{PWM} = log_2(ARR + 1)$$

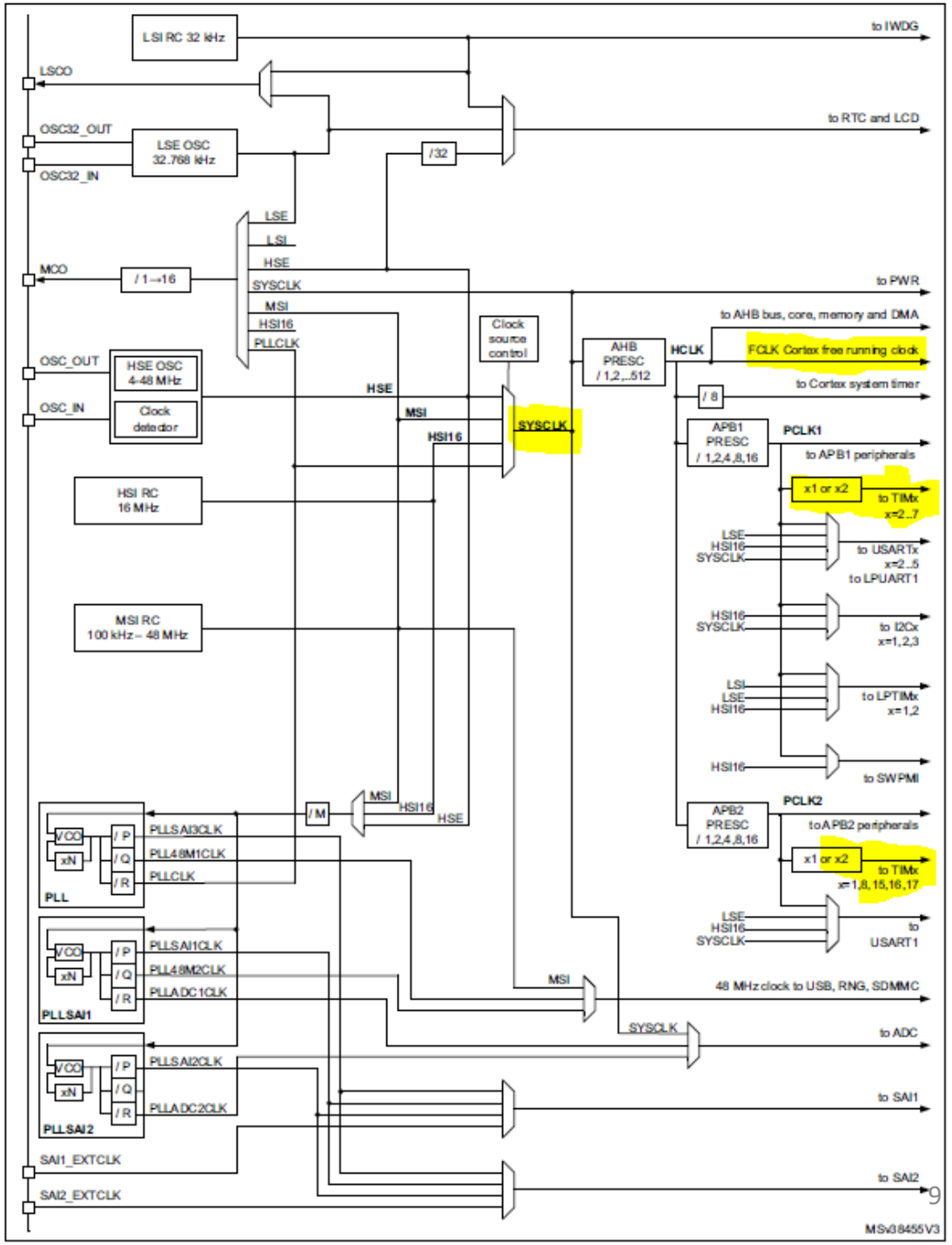
2. Resolution Control Equation

$$Resolution_{PWM} = log_2(\frac{F_{CLK}}{F_{PWM}(PSC + 1)})$$

Clock Tree

- Please see p. 208 in STM32L4 user manual

Review question:
Why does an MCU SOC need a clock tree?



SYSCLK

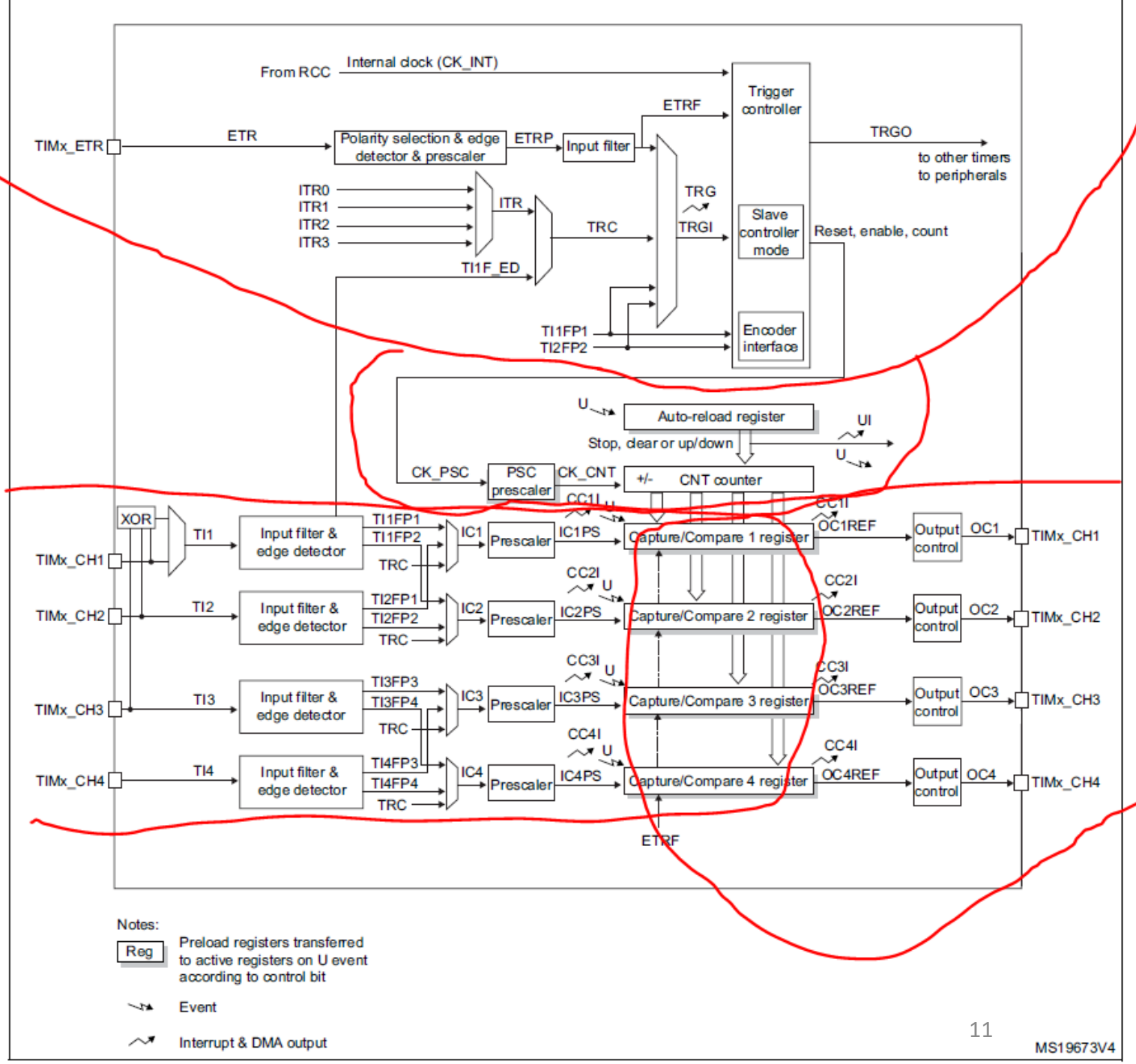
- Can be derived from HSI (High Speed Internal), HSE (High Speed External), MSI clock, Clock generated by PLL, …
- Can be used as Cortex FCLK (free running CLK), timer, peripheral control circuit

- Reference:
  - User manual of STM32L4, Section 6.2 Clocks

Review question:
Why so many clock sources are required or available in an MCU SOC?

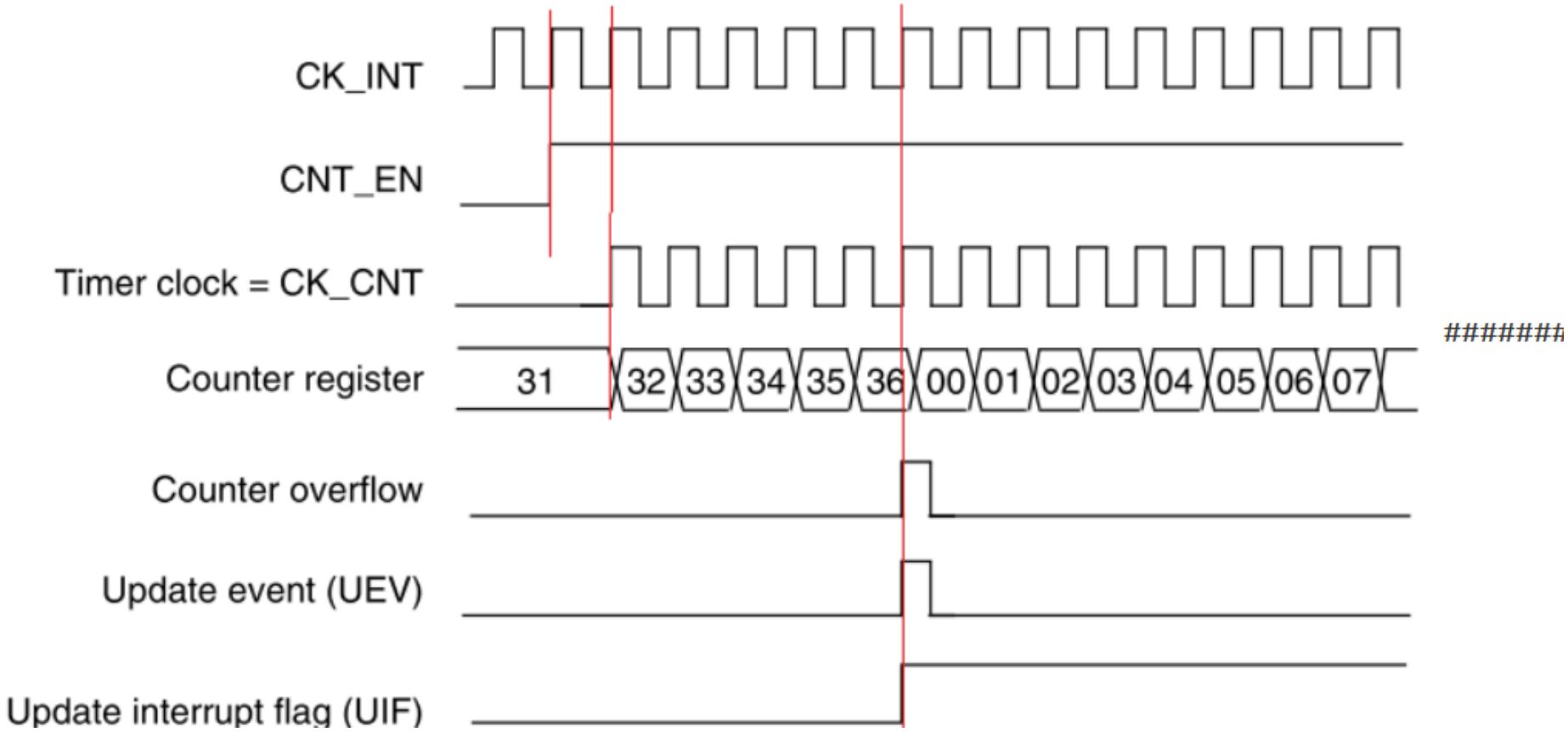
Timer

- STM32L4 user manual p.1015
- Including four parts:
  - clock source
  - timer unit
  - input capture
  - output generation

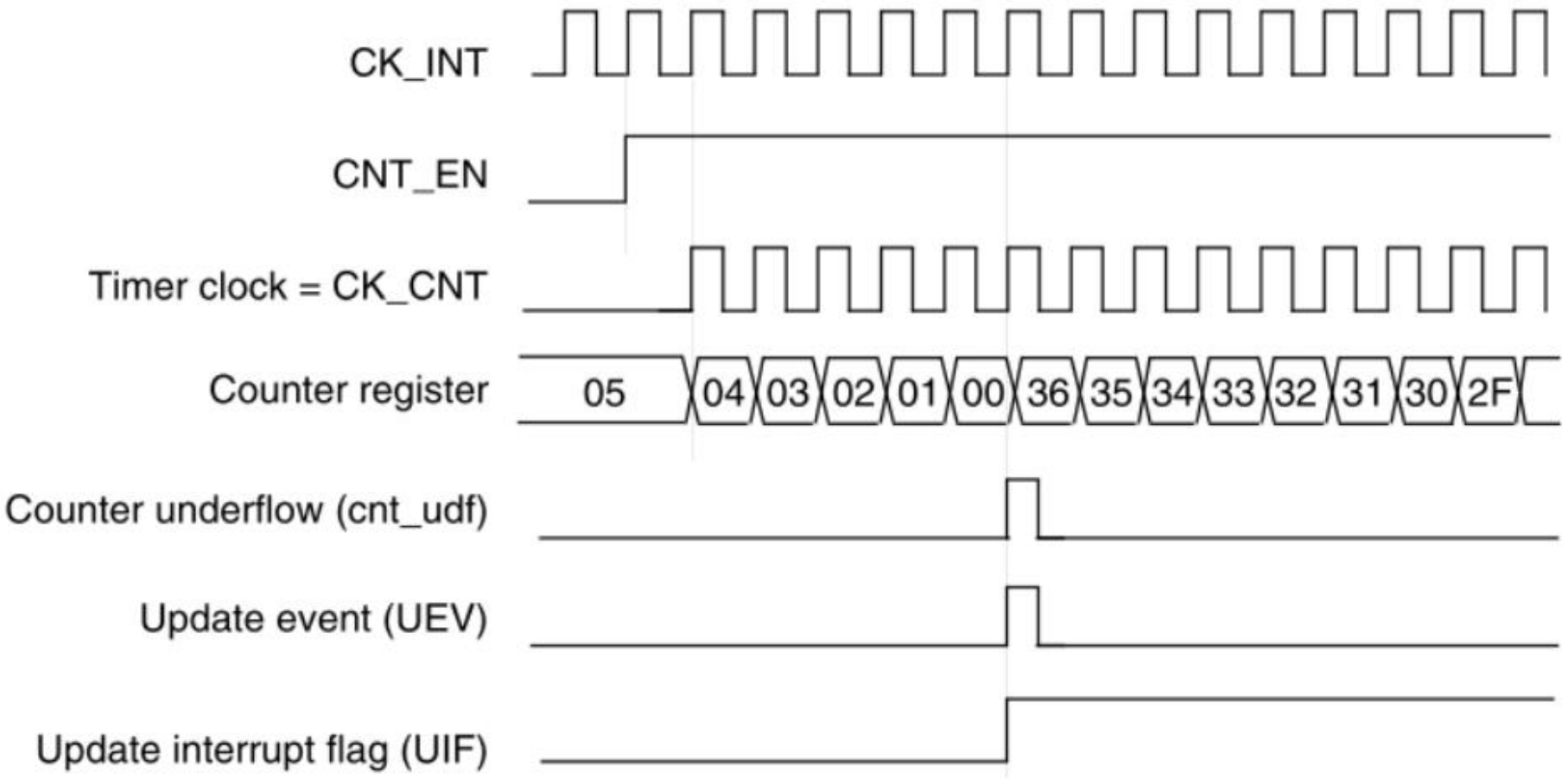


Timer Mode

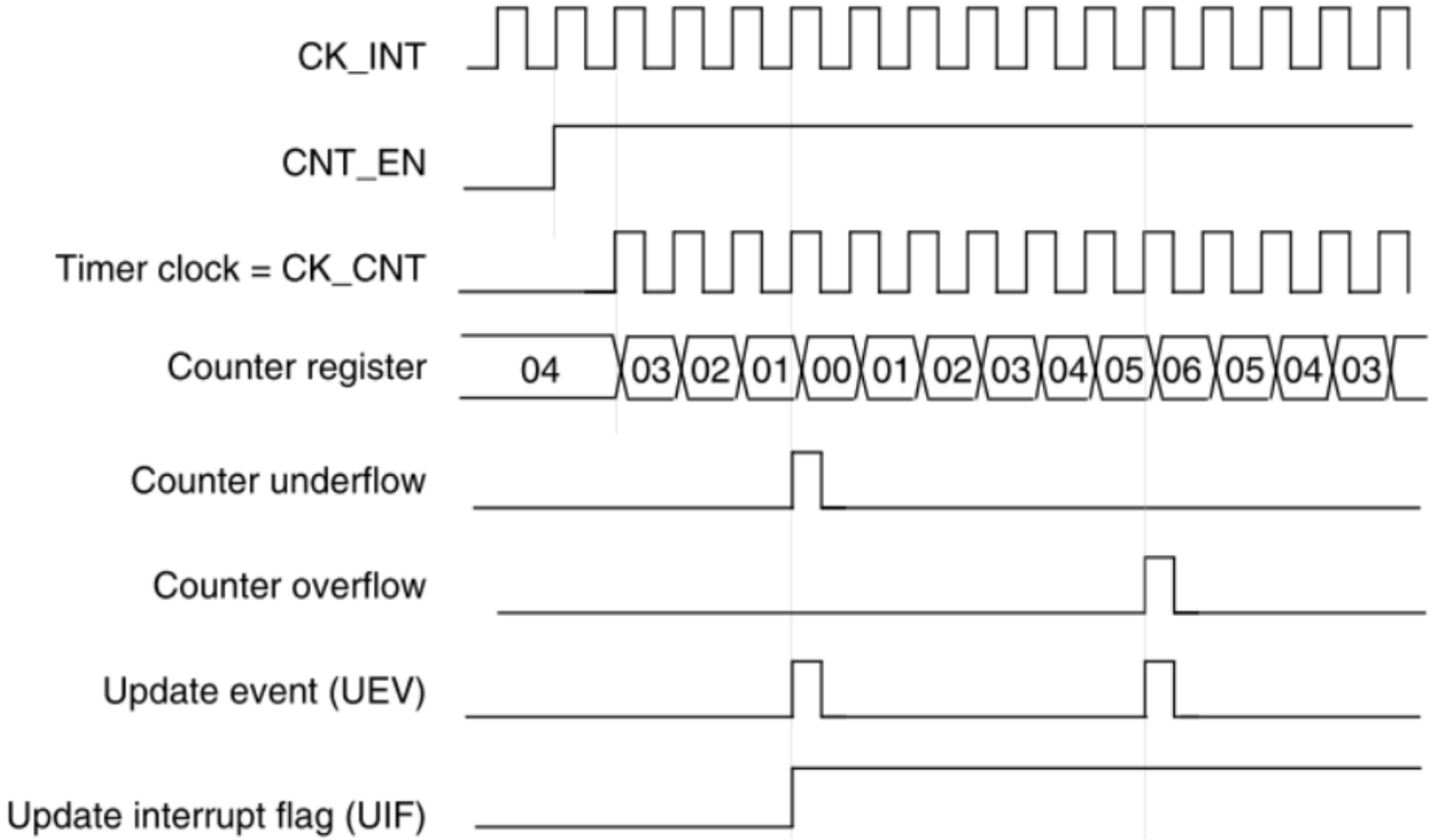
- Upcounting mode - TIMx_ARR = 0x36



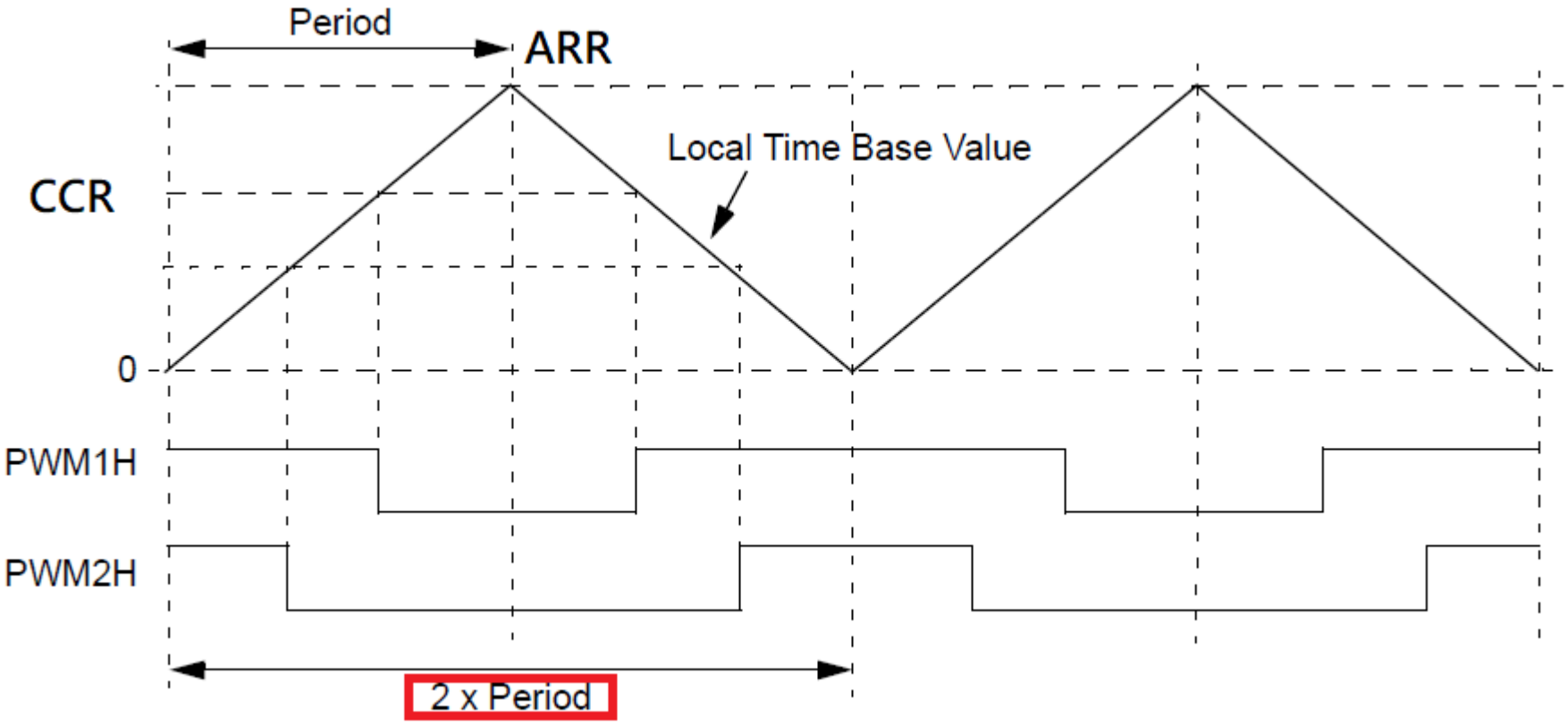
- Downcounting mode - TIMx_ARR = 0x36



Center-aligned mode - TIMx_ARR = 0x6



The effective PWM period of the center-aligned mode is **twice** the specified value



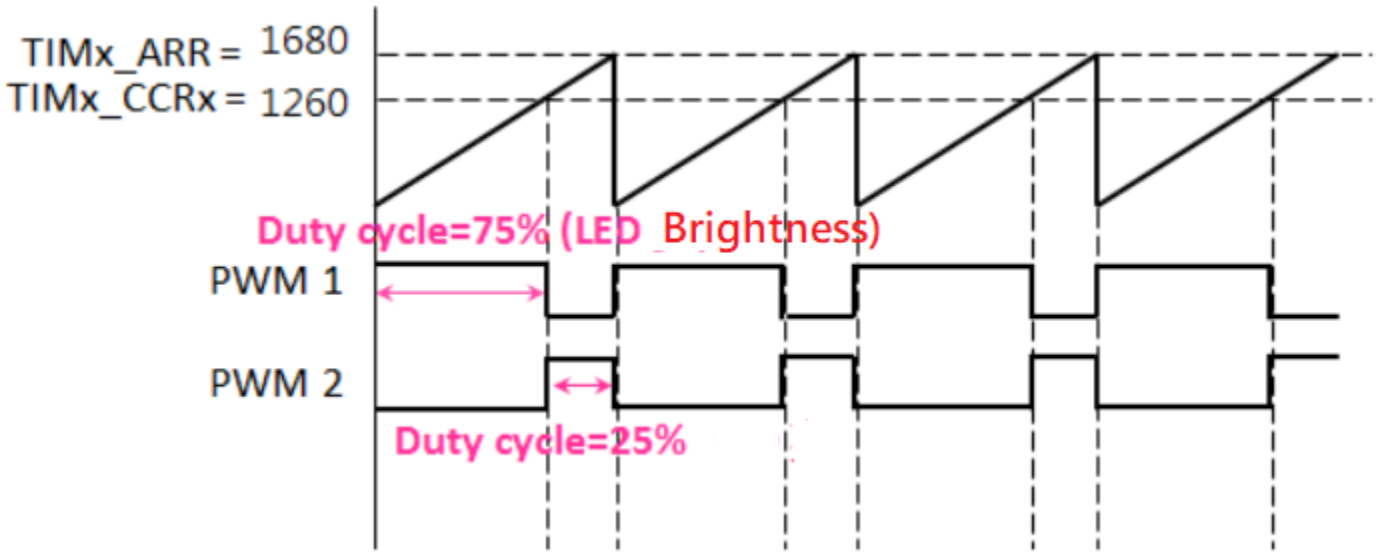
Review: Timer Setup

- CLK: The clock that drives the timer module
  - F_{CLK} is the frequency of CLK
- ARR: Auto-Reload Register
- PSC: Prescaler value
- PWM signal frequency

  $F_{CLK}/((ARR + 1)(PSC + 1))$
- PWM signal Period

  $T = (ARR + 1)(PSC + 1) / F_{CLK}$

Example: PWM Setup

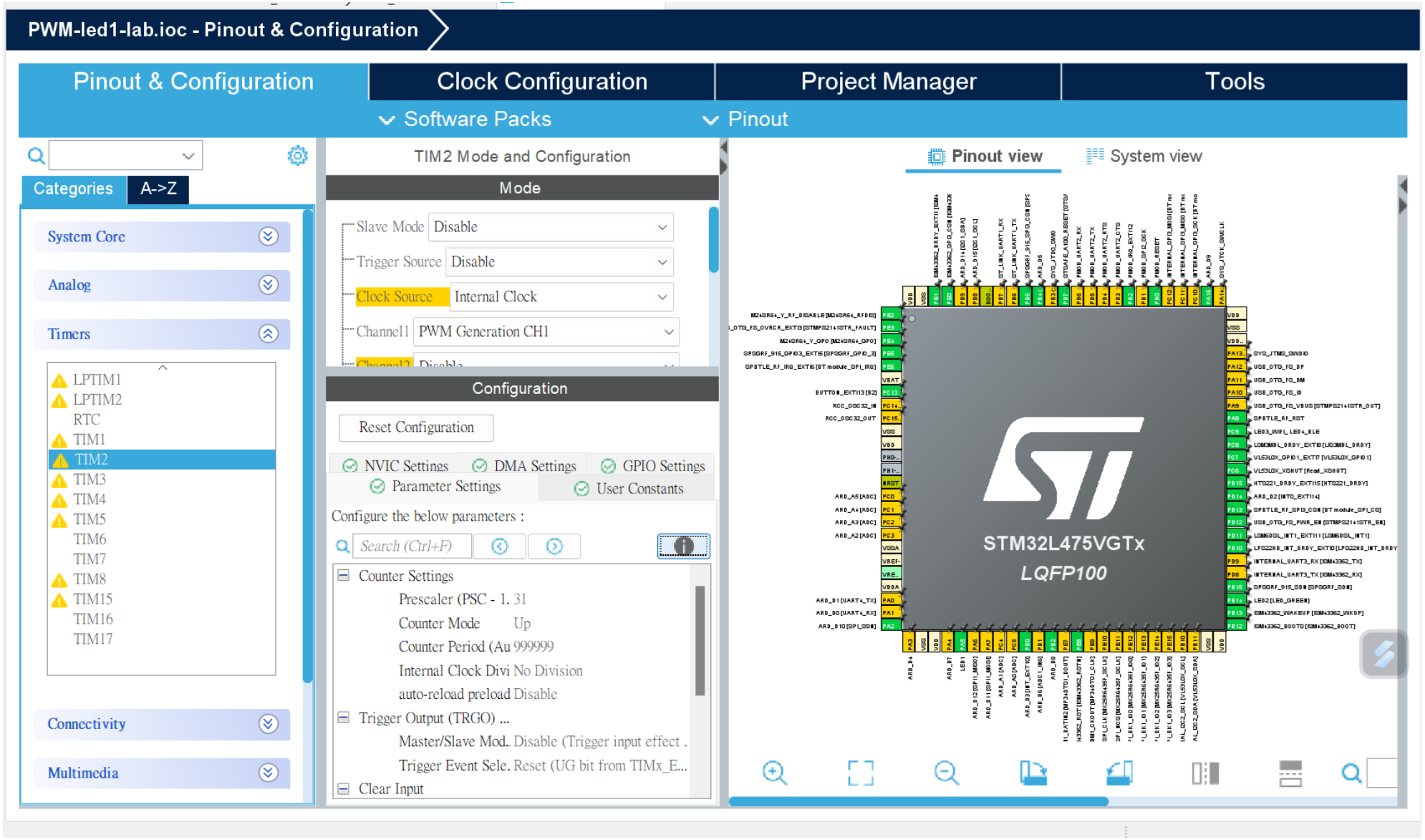


$$duty\ cycle = \frac{TIM_CCRx}{TIM_ARR}$$

Note: PWM mode 1 and PWM mode 2

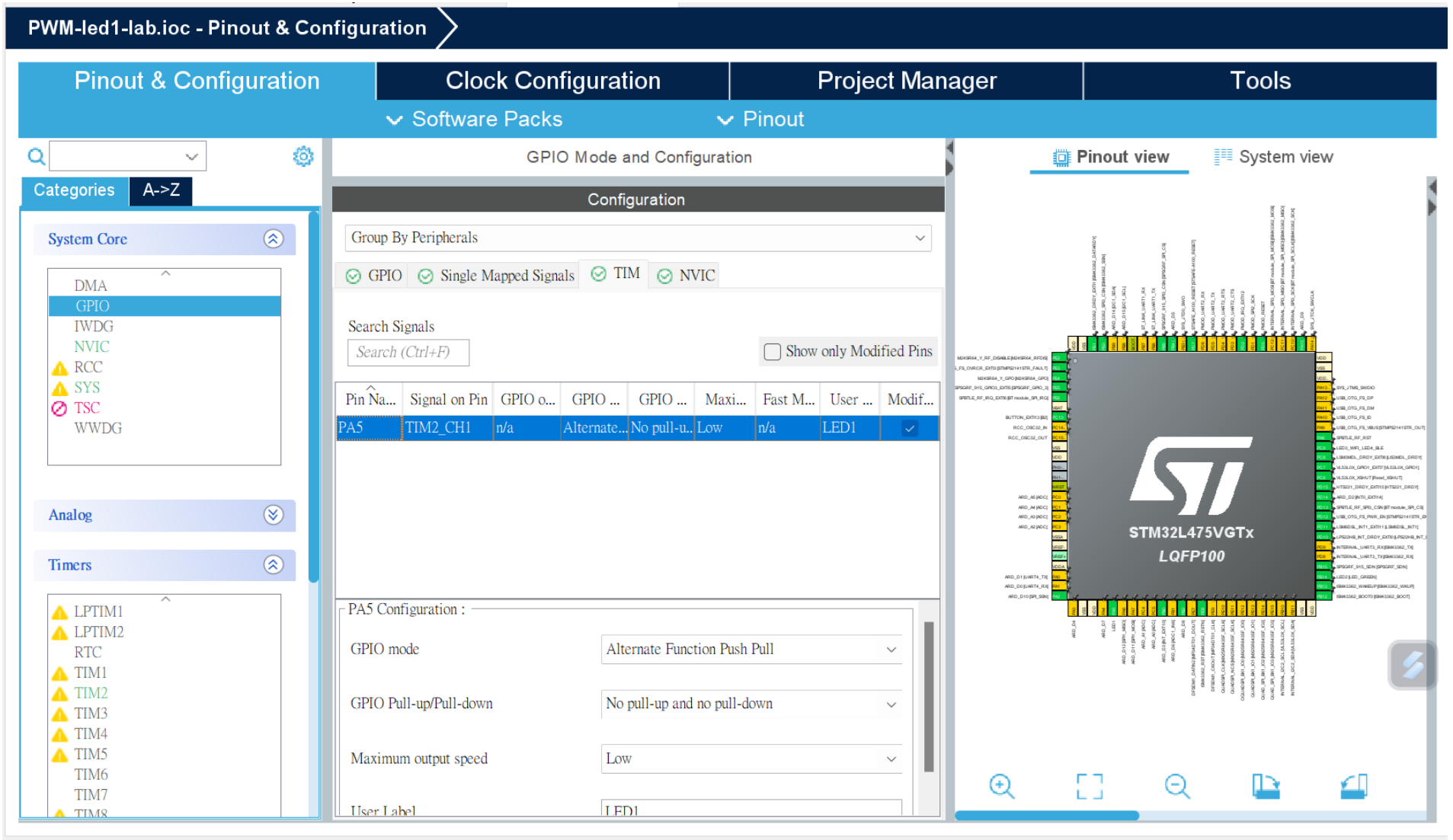
CubeMX timer setup to generate PWM output

- Note the configuration and parameter names
- What are the frequency, duty cycle, and mode of the PWM output?



GPIO setting to connect TIM2_CH1 to PA5 (LED1) or PA15 (to be observed by Logic Analyzer)

- Refer to the B-475E-IOT01A user manual for PA5 in p.29
  - LED1 is connected to PA5 of MCU
- If TIM2_CH1 is set connected PA15 using CubeMX, then we can observe the PWM waveform in CN1 pin 2 (see user manual p.31)



main.c (generated and user code editing)

- /* Private variables ---*/
- TIM_HandleTypeDef htim2;

```
int main(void)
{
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    HAL_TIM_PWM_Start(&htim2,TIM_CHANNEL_1);
    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```



Add the line of code to start PWM generation

MX_TIM2_Init()

- Note the parameters setting:

Prescaler, CounterMode, Period, ClockSource, OCMode, Pulse, …

```
169
170  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
171  TIM_MasterConfigTypeDef sMasterConfig = {0};
172  TIM_OC_InitTypeDef sConfigOC = {0};
173
174  /* USER CODE BEGIN TIM2_Init 1 */
175
176  /* USER CODE END TIM2_Init 1 */
177  htim2.Instance = TIM2;
178  htim2.Init.Prescaler = 31;
179  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
180  htim2.Init.Period = 999999;
181  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
182  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
183  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
184  {
185    Error_Handler();
186  }
187  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
188  if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
189  {
190    Error_Handler();
191  }
192  if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
193  {
194    Error_Handler();
195  }
196  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
197  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
198  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
199  {
200    Error_Handler();
201  }
202  sConfigOC.OCMode = TIM_OCMODE_PWM1;
203  sConfigOC.Pulse = 499999;
204  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
205  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
```

MX_TIM2_Init()- example

- Internal Clock 80MHz
- PSC 16-1
- Period (ARR) 1000000-1
- Pulse (CCR) 2500000-1

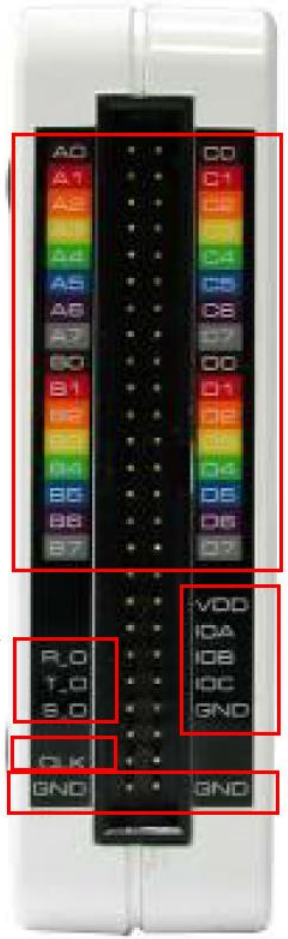
- Duty cycle 25%
- PWM frequency 5Hz

```
htim2.Instance = TIM2;
htim2.Init.Prescaler = 15;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 999999;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
  Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_O
{
  Error_Handler();
}
if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
{
  Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !
{
  Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 249999;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
```

Logic Analyzer LAP-C

LAP-C
16128

連接埠 A: A0 ~ A7
連接埠 B: B0 ~ B7
連接埠 C: C0 ~ C7
連接埠 D: D0 ~ D7



用於輸出信號以啟動其他儀器

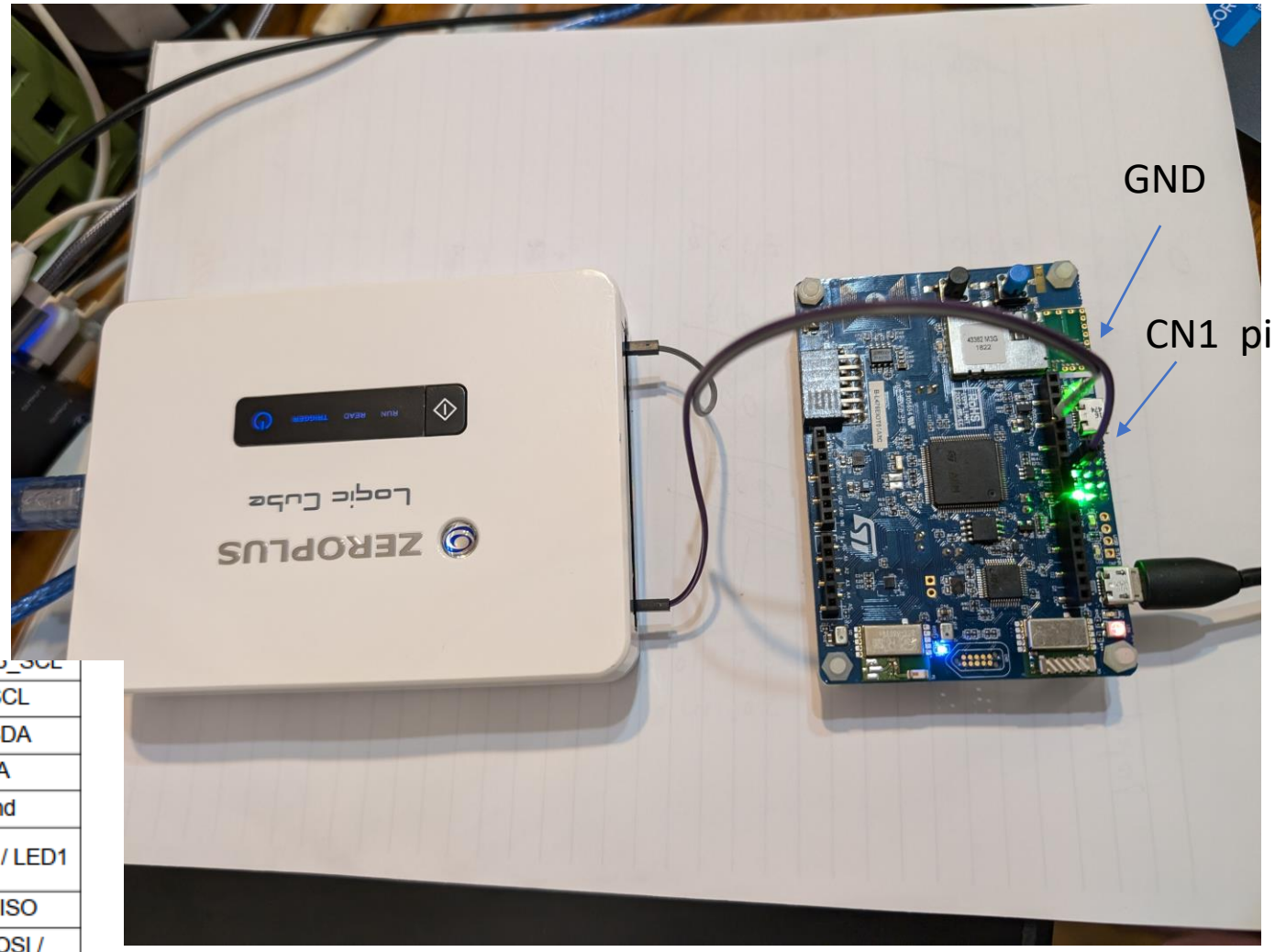
用於使用外部時脈連線

用於非指定接受分析的延伸模組或裝置

用於連接待測裝置的接地

LA-stm32 IoT node connection

- TIM2_CH1 is set connected to PA15 of the MCU via CubeMX
- PA15 is connected to CN1 pin 2 in board B-L475E-IOT01A as shown in p. 31 in the user manual



CN1	10	SCL/D15	ARD.D15-I2C1_SCL	PB8	I2C1_SCL
	9	SDA/D14	ARD.D14-I2C1_SDA	PB9	I2C1_SDA
	8	AVDD	VDDA	-	VDDA
	7	GND	GND	-	Ground
	6	SCK/D13	ARD.D13-SPI1_SCK/LED1	PA5	SPI1_SCK / LED1
	5	MISO/D12	ARD.D12-SPI1_MISO	PA6	SPI1_MISO
	4	PWM/MOSI/D11	ARD.D11-SPI1_MISO/PWM	PA7	SPI1_MOSI / TIMxx
	3	PWM/CS/D10	ARD.D10-SPI_SSN/PWM	PA2	TIM2_CH3
	2	PWM/D9	ARD.D9-PWM	PA15	TIM2_CH1
	1	D8	ARD.D8	PB2	GPIO

Logic Analyzer

- Duty cycle 25%
- PWM frequency 5Hz

